

Common supervised learning models

Process Name	Brief Description	Code Syntax
One vs One classifier (using logistic regression)	Process: This method trains one classifier for each pair of classes. Key hyperparameters:- `estimator`: Base classifier (e.g., logistic regression)Pros: Can work well for small datasets. Cons: Computationally expensive for large datasets.Common applications: Multiclass classification problems where the number of classes is relatively small.	<pre>1. from sklearn.multiclass import OneVsOneClassifier 2. from sklearn.linear_model import LogisticRegression 3. model = OneVsOneClassifier(LogisticRegr</pre>
One vs All classifier (using logistic regression)	Process: Trains one classifier per class, where each classifier distinguishes between one class and the rest. Key hyperparameters:- `estimator`: Base classifier (e.g., Logistic Regression)- `multi_class`: Strategy to handle multiclass classification (`ovr`) Pros: Simpler and more scalable than One vs One. Cons: Less accurate for highly imbalanced classes. Common applications: Common in multiclass classification problems such as image classification.	<pre>1. from sklearn.multiclass import OneVsRestClassifier 2. from sklearn.linear_model import LogisticRegression 3. model = OneVsRestClassifier(LogisticRegr</pre>

Process Name	Brief Description	Code Syntax
Decision tree classifier	Process: A tree-based classifier that splits data into smaller subsets based on feature values. Key hyperparameters:- `max_depth`: Maximum depth of the tree Pros: Easy to interpret and visualize. Cons: Prone to overfitting if not pruned properly. Common applications: Classification tasks, such as credit risk assessment.	<pre>1. from sklearn.tree import DecisionTreeClassifier 2. model = DecisionTreeClassifier(max_depth=</pre>
Decision tree regressor	Process: Similar to the decision tree classifier, but used for regression tasks to predict continuous values. Key hyperparameters:- `max_depth`: Maximum depth of the tree Pros: Easy to interpret, handles nonlinear data. Cons: Can overfit and perform poorly on noisy data. Common applications: Regression tasks, such as predicting housing prices.	<pre>1. from sklearn.tree import DecisionTreeRegressor 2. model = DecisionTreeRegressor(max_depth=</pre>
Linear SVM classifier	Process: A linear classifier that finds the optimal hyperplane separating classes with a maximum margin. Key hyperparameters:- `C`:	<pre>1. from sklearn.svm import SVC 2. model = SVC(kernel='linear', C=1</pre>

Process Name	Brief Description	Code Syntax
	Regularization parameter- `kernel`: Type of kernel function ('linear', 'poly', 'rbf', etc.)- `gamma`: Kernel coefficient (only for 'rbf', 'poly', etc.)Pros: Effective for high-dimensional spaces.Cons: Not ideal for nonlinear problems without kernel tricks.Common applications: Text classification and image recognition.	
K-nearest neighbors classifier	Process: Classifies data based on the majority class of its nearest neighbors.Key hyperparameters:- `n_neighbors`: Number of neighbors to use- `weights`: Weight function used in prediction ('uniform' or 'distance')- `algorithm`: Algorithm used to compute the nearest neighbors ('auto', 'ball_tree', 'kd_tree', 'brute')Pros: Simple and effective for small datasets.Cons: Computationally expensive as the dataset grows.Common applications: Recommendation systems, image recognition.	<pre>1. from sklearn.neighbors import KNeighborsClassifier 2. model = KNeighborsClassifier(n_neighbors= weights='uniform')</pre>
Random Forest regressor	Process: An ensemble method using multiple decision trees to improve accuracy and reduce overfitting.Key hyperparameters:- `n_estimators`: Number of trees	<pre>1. from sklearn.ensemble import RandomForestRegressor 2. model = RandomForestRegressor(n_estimators= max_depth=5)</pre>

Process Name	Brief Description	Code Syntax
	in the forest- <code>`max_depth`</code>: Maximum depth of each tree Pros: Less prone to overfitting than individual decision trees. Cons: Model complexity increases with the number of trees. Common applications: Regression tasks such as predicting sales or stock prices.	
XGBoost regressor	Process: A gradient boosting method that builds trees sequentially to correct errors from previous trees. Key hyperparameters:- <code>`n_estimators`</code>: Number of boosting rounds- <code>`learning_rate`</code>: Step size to improve accuracy- <code>`max_depth`</code>: Maximum depth of each tree Pros: High accuracy and works well with large datasets. Cons: Computationally intensive, complex to tune. Common applications: Predictive modeling, especially in Kaggle competitions.	<pre>1. import xgboost as xgb 2. model = xgb.XGBRegressor(n_estimators=10 learning_rate=0.1, max_depth=5)</pre>

Associated functions used

Method Name	Brief Description	Code Syntax
OneHotEncoder	Transforms categorical features into a one-hot encoded matrix.	<code>from sklearn.preprocessing import OneHotEncoder</code> <code>encoder = OneHotEncoder(sparse=Fa</code> <code>encoded_data = encoder.fit_transform(categorical,</code>
accuracy_score	Computes the accuracy of a classifier by comparing predicted and true labels.	<code>from sklearn.metrics import accur</code> <code>accuracy = accuracy_score(y_true,</code>
LabelEncoder	Encodes labels (target variable) into numeric format.	<code>from sklearn.preprocessing import</code> <code>encoder = LabelEncoder()</code> <code>encoded_labels = encoder.fit_tran</code>
plot_tree	Plots a decision tree model for visualization.	<code>from sklearn.tree import plot_tre</code> <code>plot_tree(model, max_depth=3, fil</code>
normalize	Scales each feature to have zero mean and unit variance (standardization).	<code>from sklearn.preprocessing import</code> <code>normalized_data = normalize(data,</code>
compute_sample_weight	Computes sample weights for imbalanced datasets.	<code>from sklearn.utils.class_weight i</code> <code>compute_sample_weight</code> <code>weights = compute_sample_weight(class_weigh</code> <code>y=y)</code>
roc_auc_score	Computes the Area Under the	<code>from sklearn.metrics import roc_a</code>

Method Name	Brief Description	Code Syntax
	Receiver Operating Characteristic Curve (AUC-ROC) for binary classification models.	2. <code>auc = roc_auc_score(y_true, y_sco</code>